

08 - pi-mono 项目深度分析及与 oh-my-opencode / OpenCode 对比

目录

- 第一部分：pi-mono 架构全景
- 第二部分：各包模块详细分析
- 第三部分：核心实现流程深入剖析
- 第四部分：与 oh-my-opencode / OpenCode 详细对比

第一部分：pi-mono 架构全景

1.1 项目概况

pi-mono（品牌名："pi"）是由 Mario Zechner（badlogic，libGDX 创始人）开发的开源 AI 编程助手。它是一个完整的独立产品，从 LLM API 抽象层到终端 UI 全部自研。

属性	值
GitHub Stars	~17.5k
语言比例	96.5% TypeScript
许可证	MIT
版本	v0.55.1+
构建	npm workspaces monorepo
测试	vitest
风格检查	biome
版本策略	lockstep（所有包同版本）

1.2 Monorepo 7 个包

```
pi-mono/
├── packages/
│   ├── ai/                # @mariozechner/pi-ai - 统一多提供商 LLM API
│   ├── agent/             # @mariozechner/pi-agent-core - 最小化 Agent 运行时
│   ├── coding-agent/      # @mariozechner/pi-coding-agent - 完整编码代理 CLI
│   ├── tui/               # @mariozechner/pi-tui - 终端 UI 框架
│   ├── web-ui/            # Web 组件 UI
│   ├── mom/               # Slack Bot (委托给 pi coding agent)
│   └── pods/              # vLLM 部署管理 CLI
```

关键设计理念：分层解耦。每一层可独立使用——你可以只用 pi-ai 调 LLM，或只用 pi-agent-core 跑 Agent 循环，而无需引入编码代理或 TUI。

1.3 哲学：极简可组合 + 激进可扩展

pi 的核心哲学在 README 中清晰阐述：

"Pi is aggressively extensible so it doesn't have to dictate your workflow."

这意味着 pi 有意**不内置**以下功能（对比 Claude Code / oh-my-opencode 均内置）：

功能	pi 的做法	理由
子代理 (Sub-agents)	不内置，用 Extension 或 tmux	"There's many ways to do this"
Plan/Build 模式	不内置，用 Extension	"Write plans to files"
MCP 协议	不内置，用 Extension	"Build CLI tools with READMEs"
权限弹窗	不内置，用 Extension	"Run in a container"
后台 Bash	不内置，用 tmux	"Full observability"
TODO 追踪	不内置，用文件	"They confuse models"

第二部分：各包模块详细分析

2.1 @mariozechner/pi-ai — 统一 LLM API 层

目录结构：

```
packages/ai/src/
├── types.ts           # 核心类型定义
├── stream.ts          # Stream 编排入口
├── models.ts          # Model Registry (运行时模型查找)
├── models.generated.ts # 自动生成的模型数据库
├── api-registry.ts    # API Provider 注册表
├── env-api-keys.ts    # 环境变量 API Key 解析
├── cli.ts             # CLI 工具
├── index.ts           # 桶导出
├── providers/         # 各提供商实现
│   ├── anthropic-messages.ts
│   ├── openai-completions.ts
│   ├── openai-responses.ts
│   ├── google-generative-ai.ts
│   ├── google-vertex.ts
│   ├── bedrock-converse-stream.ts
│   └── register-builtins.ts
└── utils/
    ├── event-stream.ts # EventStream<E, R> 异步迭代流
    └── http-proxy.ts   # HTTP 代理支持
```

2.1.1 types.ts — 核心类型系统

pi-ai 定义了统一的消息/模型/工具类型，支持所有提供商：

```

// API 类型 (决定用哪个提供商适配器)
type KnownApi = "openai-completions" | "openai-responses" | "anthropic-messages"
  | "google-generative-ai" | "bedrock-converse-stream" | ...

// 提供商类型 (决定基础 URL + API Key)
type KnownProvider = "anthropic" | "openai" | "google" | "amazon-bedrock"
  | "github-copilot" | "xai" | "groq" | "openrouter" | ...

// 消息类型
interface UserMessage { role: "user"; content: string | (TextContent | ImageContent)[]; }
interface AssistantMessage { role: "assistant"; content: (TextContent | ThinkingContent)[]; }
interface ToolResultMessage { role: "toolResult"; toolCallId: string; content: (TextCon

// 模型接口 (核心——所有操作围绕 Model 对象)
interface Model<TApi extends Api> {
  id: string; name: string; api: TApi; provider: Provider;
  baseUrl: string; reasoning: boolean; input: ("text" | "image")[];
  cost: { input: number; output: number; cacheRead: number; cacheWrite: number };
  contextWindow: number; maxTokens: number;
  compat?: OpenAICompletionsCompat | OpenAIResponsesCompat; // 兼容性覆盖
}

// 工具定义 (使用 TypeBox schema)
interface Tool<TParameters extends TSchema = TSchema> {
  name: string; description: string; parameters: TParameters;
}

// 流式事件——细粒度流
type AssistantMessageEvent =
  | { type: "start"; partial: AssistantMessage }
  | { type: "text_delta"; contentIndex: number; delta: string; partial: AssistantMessag
  | { type: "thinking_delta"; contentIndex: number; delta: string; partial: AssistantMe
  | { type: "toolcall_start" | "toolcall_delta" | "toolcall_end"; ... }
  | { type: "done"; reason: StopReason; message: AssistantMessage }
  | { type: "error"; reason: StopReason; error: AssistantMessage }

```

独特设计:

- **OpenAICompletionsCompat**: pi 对 OpenAI 兼容 API 有极其精细的兼容性控制 (supportsStore, supportsDeveloperRole, supportsReasoningEffort, requiresAssistantAfterToolResult, requiresThinkingAsText, requiresMistralToolIds, thinkingFormat 等), 这是从大量实际使用中沉淀出来的

- **ThinkingLevel**: "minimal" | "low" | "medium" | "high" | "xhigh" ——比 Claude Code 的 thinking budget 更精细
- **CacheRetention**: "none" | "short" | "long" ——显式控制 prompt cache 保留策略
- **Transport**: "sse" | "websocket" | "auto" ——支持 WebSocket 传输（如 OpenAI Codex）

2.1.2 stream.ts — 流编排

极简的 4 个函数入口：

```
function stream(model, context, options?)           // 底层流, 返回 EventStream
function complete(model, context, options?)         // 底层完成 (await stream.result())
function streamSimple(model, context, options?)     // 带 reasoning 参数的简化版
function completeSimple(model, context, options?)   // 简化完成
```

stream() 基于 api-registry 查找提供商，调用提供商的 stream() 方法。streamSimple() 在此基础上加了 thinking level 映射。

2.1.3 models.ts — 模型注册表

```
const modelRegistry: Map<string, Map<string, Model<Api>>> = new Map();
// provider → modelId → Model

function getModel(provider, modelId): Model // 按 provider + id 获取
function getModels(provider): Model[]       // 获取一个 provider 的所有模型
function getProviders(): KnownProvider[]    // 获取所有 provider
function calculateCost(model, usage): Cost  // 计算费用
function supportsXhigh(model): boolean     // 是否支持 xhigh thinking
```

模型数据存储在 models.generated.ts（自动生成），包含每个 provider 的所有模型及其完整配置。

2.2 @mariozechner/pi-agent-core — 最小化 Agent 运行时

这是 pi 架构中最精妙的部分——仅 5 个文件实现了完整的 Agent 循环。

```
packages/agent/src/
├─ types.ts           # AgentState, AgentTool, AgentEvent, AgentLoopConfig 等
├─ agent.ts           # Agent 类（状态机 + 事件发射 + 消息队列）
├─ agent-loop.ts      # Agent 循环（LLM 调用 → 工具执行 → 继续判断）
├─ proxy.ts           # 代理支持
└─ index.ts           # 桶导出
```

2.2.1 types.ts — Agent 类型

重要设计：AgentMessage 的可扩展性

```
// 声明合并允许应用添加自定义消息类型
interface CustomAgentMessages {
  // 应用通过 declaration merging 扩展
}

type AgentMessage = Message | CustomAgentMessages[keyof CustomAgentMessages];

// Agent 状态
interface AgentState {
  systemPrompt: string;
  model: Model<any>;
  thinkingLevel: ThinkingLevel;
  tools: AgentTool<any>[];
  messages: AgentMessage[];
  isStreaming: boolean;
  streamMessage: AgentMessage | null;
  pendingToolCalls: Set<string>;
  error?: string;
}
```

AgentLoopConfig 是 Agent 循环的核心配置：

```
interface AgentLoopConfig extends SimpleStreamOptions {
  model: Model<any>;
  convertToLlm: (messages: AgentMessage[]) => Message[] | Promise<Message[]>;
  transformContext?: (messages: AgentMessage[], signal?) => Promise<AgentMessage[]>;
  getApiKey?: (provider: string) => Promise<string | undefined> | string | undefined;
  getSteeringMessages?: () => Promise<AgentMessage[]>; // 中断消息
  getFollowUpMessages?: () => Promise<AgentMessage[]>; // 后续消息
}
```

这个设计的精妙在于：

- **convertToLlm**：消息「飞越」LLM 边界时的转换函数——应用可以有自定义消息类型，只在发给 LLM 时转换
- **transformContext**：Context window 管理的钩子（裁剪旧消息、注入外部上下文）
- **getSteeringMessages**：用户中断机制——在工具执行后检查是否有新消息插入

- **getFollowUpMessages**: Agent 完成后的后续消息——实现了排队发送

2.2.2 agent-loop.ts — Agent 循环实现

这是 pi 的核心循环，结构如下：

```
agentLoop(prompts, context, config, signal, streamFn)
└─ runLoop(context, newMessages, config, signal, stream, streamFn)
    │   └─ 外循环: while(true) - 处理 followUp 消息
    │       └─ 内循环: while(hasMoreToolCalls || pendingMessages.length > 0) - 处理工具和
    │           │   └─ 处理 pendingMessages (steering 消息注入)
    │           │   └─ streamAssistantResponse() - 调 LLM
    │           │       │   └─ config.transformContext(messages) - 上下文转换
    │           │       │   └─ config.convertToLlm(messages) - 转为 LLM 消息
    │           │       └─ streamFunction(model, llmContext, options) - 实际 API 调用
    │           └─ executeToolCalls() - 顺序执行工具
    │               │   └─ validateToolArguments(tool, toolCall)
    │               │   └─ tool.execute(id, args, signal, onUpdate)
    │               └─ 每个工具后检查 getSteeringMessages()
    └─ 检查 steering 消息
    └─ 检查 getFollowUpMessages() - 有则继续外循环
```

工具执行的 Steering 中断是一个精巧设计：当用户在 Agent 执行工具时发送新消息，剩余工具会被标记为 "Skipped due to queued user message" 并跳过，新消息立即注入上下文。

2.2.3 agent.ts — Agent 类

Agent 类是一个**状态机 + 事件发射器 + 消息队列管理器**：

```

class Agent {
    // 状态
    private _state: AgentState;
    private listeners: Set<(e: AgentEvent) => void>;
    private steeringQueue: AgentMessage[];    // Steering 消息队列
    private followUpQueue: AgentMessage[];    // FollowUp 消息队列

    // 核心方法
    prompt(message)           // 发送提示 (启动 Agent 循环)
    continue()                // 从当前上下文继续 (重试/恢复)
    steer(message)            // 中断—注入 Steering 消息
    followUp(message)         // 等待完成后—注入 FollowUp 消息
    abort()                   // 取消当前操作

    // 队列模式
    steeringMode: "all" | "one-at-a-time"    // 一次全发 vs 一个一个发
    followUpMode: "all" | "one-at-a-time"
}

```

2.3 @mariozechner/pi-coding-agent — 完整编码代理

这是最大的包，也是面向用户的产品。


```
packages/coding-agent/src/
├─ cli.ts / main.ts      # CLI 入口 + 主流程
├─ config.ts             # 配置管理
├─ migrations.ts         # 配置迁移
├─ core/                 # 核心逻辑 (25+ 文件)
│   ├─ agent-session.ts  # AgentSession 类 (~2000行, 核心中的核心)
│   ├─ system-prompt.ts  # 系统提示构建
│   ├─ model-resolver.ts  # 模型解析
│   ├─ model-registry.ts  # 模型注册 + API Key 管理
│   ├─ session-manager.ts # 会话持久化 (JSONL 树结构)
│   ├─ settings-manager.ts # 设置管理
│   ├─ bash-executor.ts   # Bash 执行器
│   ├─ event-bus.ts       # 事件总线
│   ├─ keybindings.ts     # 键绑定管理
│   ├─ prompt-templates.ts # 提示模板
│   ├─ skills.ts          # Skills 系统
│   ├─ slash-commands.ts  # 斜杠命令
│   ├─ resource-loader.ts # 资源加载器
│   ├─ exec.ts            # 子进程执行
│   ├─ sdk.ts             # SDK 入口
│   ├─ compaction/        # 上下文压缩
│   ├─ extensions/        # 扩展系统
│   │   ├─ types.ts       # 扩展 API 类型 (~1200行)
│   │   ├─ loader.ts      # 扩展加载器
│   │   ├─ runner.ts      # 扩展运行器
│   │   └─ wrapper.ts     # 工具包装器
│   ├─ tools/             # 内置工具
│   │   ├─ bash.ts        # Bash 工具
│   │   ├─ read.ts        # 文件读取
│   │   ├─ write.ts       # 文件写入
│   │   ├─ edit.ts        # 文件编辑 (精确替换)
│   │   ├─ edit-diff.ts   # 差异编辑 (模糊匹配)
│   │   ├─ grep.ts        # 搜索
│   │   ├─ find.ts        # 文件查找
│   │   ├─ ls.ts          # 目录列表
│   │   └─ path-utils.ts  # 路径工具
│   └─ export-html/       # HTML 导出
├─ modes/                # 运行模式
│   ├─ interactive/      # 交互模式 (TUI)
│   ├─ print/             # 打印模式 (非交互)
│   └─ rpc/               # RPC 模式 (进程间通信)
```

— cli/	# CLI 子命令
— utils/	# 工具函数

2.3.1 agent-session.ts — 核心会话管理（~2000行）

`AgentSession` 是 `pi` 最重要的类。它创建在 `Agent` 之上，封装了编码代理的全部生命周期管理：

AgentSession

- └─ 事件订阅 (Event Subscription)
 - └─ 内部处理 agent 事件 → 持久化到 session
 - └─ 转发到 extension runner
 - └─ 发射到外部监听器
- └─ 提示处理 (Prompting)
 - └─ Extension 命令拦截 (/command)
 - └─ Input 事件 (Extension 可拦截/转换)
 - └─ Skill 命令展开 (/skill:name)
 - └─ Prompt Template 展开
 - └─ Streaming 时排队 (steer/followUp)
 - └─ before_agent_start 扩展事件
 - └─ agent.prompt(messages)
- └─ 模型管理 (Model Management)
 - └─ setModel() - 验证 API Key + 保存
 - └─ cycleModel() - Ctrl+P 循环切换
 - └─ 思维级别管理
- └─ 上下文压缩 (Compaction)
 - └─ 手动压缩 (/compact)
 - └─ 自动压缩 (阈值触发 / 溢出恢复)
 - └─ Extension 可替换压缩逻辑
- └─ 自动重试 (Auto-Retry)
 - └─ 指数退避
 - └─ 识别可重试错误 (overloaded, rate_limit, 5xx)
 - └─ 上下文溢出重试 (交给 Compaction)
- └─ Bash 执行 (用户 ! / !! 命令)
- └─ 会话管理 (Session Management)
 - └─ newSession() / switchSession()
 - └─ fork() - 创建分支
 - └─ navigateTree() - 树导航
- └─ 扩展系统 (Extensions)
 - └─ Extension Runner 生命周期
 - └─ 工具注册 + 包装
 - └─ 资源发现
- └─ 运行时构建 (_buildRuntime)
 - └─ 创建内置工具
 - └─ 加载 Extensions
 - └─ 注册 Extension 工具
 - └─ 包装工具 (Extension 可拦截)
 - └─ 构建系统提示

2.3.2 Extensions 系统（核心差异化）

pi 的 Extension 系统是其最重要的差异化特性。它合并了其他工具中的 hooks + custom tools + plugins 概念：

ExtensionAPI（传给扩展工厂函数的接口）：

```
interface ExtensionAPI {
  // 事件订阅（26 种事件类型）
  on("resources_discover", handler)
  on("session_start" | "session_switch" | "session_fork" | ..., handler)
  on("context", handler)           // 可修改发送给 LLM 的消息
  on("before_agent_start", handler) // 可修改系统提示 + 注入消息
  on("agent_start" | "agent_end", handler)
  on("turn_start" | "turn_end", handler)
  on("message_start" | "message_update" | "message_end", handler)
  on("tool_execution_start" | "tool_execution_end", handler)
  on("tool_call", handler)         // 可阻止工具执行
  on("tool_result", handler)       // 可修改工具结果
  on("model_select", handler)
  on("user_bash", handler)         // 可替换 bash 执行
  on("input", handler)            // 可拦截/转换用户输入

  // 注册
  registerTool(tool)               // 注册 LLM 可调用的工具
  registerCommand(name, options)   // 注册 /command
  registerShortcut(key, options)   // 注册键盘快捷键
  registerFlag(name, options)      // 注册 CLI flag
  registerMessageRenderer(type, fn) // 自定义消息渲染
  registerProvider(name, config)   // 注册 LLM 提供商（含 OAuth）

  // 操作
  sendMessage(message, options)    // 发送自定义消息
  sendUserMessage(content, options) // 发送用户消息（触发 turn）
  setActiveTools(toolNames)        // 设置活跃工具
  setModel(model)                  // 切换模型
  setThinkingLevel(level)          // 设置思维级别
  setSessionName(name)             // 设置会话名
  exec(command, args, options)     // 执行 shell 命令
  events: EventBus                 // 扩展间通信
}
```

ExtensionUIContext（UI 操作原语——极其丰富）：

```
interface ExtensionUIContext {
    select(title, options)      // 选择对话框
    confirm(title, message)     // 确认对话框
    input(title, placeholder)   // 输入对话框
    notify(message, type)       // 通知
    editor(title, prefill)       // 多行编辑器
    custom(factory, options)     // 任意自定义 TUI 组件 (含 overlay)

    setStatus(key, text)        // 状态栏文本
    setWorkingMessage(message)  // 工作中提示
    setWidget(key, content)     // 编辑器上方/下方 widget
    setFooter(factory)          // 自定义页脚
    setHeader(factory)          // 自定义页头
    setEditorComponent(factory) // 替换编辑器 (可实现 Vim 模式)
    setTitle(title)             // 终端标题
    setTheme(theme)             // 运行时换肤

    onTerminalInput(handler)    // 原始终端输入监听
    pasteToEditor(text)         // 编程粘贴
    setEditorText(text)         // 设置编辑器内容
    getEditorText()             // 获取编辑器内容
}
```

2.3.3 内置工具

pi 默认只启用 4 个工具： read , bash , edit , write 。可选额外启用 grep , find , ls 。

工具	文件	功能
bash	bash.ts	执行 Shell 命令，含 spawn hook
read	read.ts	读取文件内容，支持图片自动缩放
write	write.ts	创建/覆写文件
edit	edit.ts	精确文本替换（find & replace），含 edit-diff.ts 模糊匹配
grep	grep.ts	内容搜索（ripgrep 风格）
find	find.ts	文件查找（glob 模式）
ls	ls.ts	目录列表

关键设计：工具数量极少（7个），且默认只启用 4 个。pi 认为**less is more**——所有高级功能通过 Extension 添加。

2.3.4 system-prompt.ts — 系统提示

```
function buildSystemPrompt(options: BuildSystemPromptOptions): string
```

系统提示的构成：

- 1. 角色定义："You are an expert coding assistant operating inside pi"
- 2. 工具列表（仅列出启用的工具 + 描述）
- 3. 使用指南（根据启用的工具动态生成——如有 grep 则提示 prefer grep over bash）
- 4. Pi 文档引用（docs/ 路径，仅在用户问关于 pi 时读取）
- 5. 附加系统提示（用户指定）
- 6. 项目上下文文件（AGENTS.md 等）
- 7. Skills 部分
- 8. 当前时间 + 工作目录

关键差异：pi 的系统提示**短而精**（对比 Claude Code 的万字系统提示）。它只告知 LLM 有哪些工具、基本指南、和项目上下文，其余靠模型自身能力。

2.4 @mariozechner/pi-tui — 终端 UI 框架

这是 pi 的一个**独立且通用的 TUI 框架**——不依赖 pi 的任何其他包。

核心特性：

特性	说明
差异渲染	3 策略：首次全渲染 / 宽度变化全渲染 / 正常增量更新
同步输出	CSI 2026 原子屏幕更新（无闪烁）
组件化	统一 Component 接口：render(width): string[]
Overlay	支持锚定定位、百分比定位、margin、响应式可见性
Focusable	IME 支持（CJK 输入法定位）
主题	热重载主题系统

内置组件：

组件	功能
Text	多行文本 + 自动折行
TruncatedText	单行截断文本
Input	单行输入（水平滚动）
Editor	多行编辑器（自动补全、文件补全、粘贴处理、垂直滚动）
Markdown	Markdown 渲染（语法高亮、代码块、链接）
Loader / CancellableLoader	加载动画
SelectList	交互式选择列表
SettingsList	设置面板
Image	内联图片（Kitty/iTerm2 协议）
Box / Container / Spacer	布局组件

****设计哲学：****pi-tui 完全基于"每一帧返回字符串数组"的简单模型——每个组件的 `render(width)` 返回行数组。框架负责差异比较和最小化终端输出。这比 React-like 虚拟 DOM 方案更底层但性能更好。

2.5 @mariozechner/pi-web-ui — Web UI

Web 组件库，用于在浏览器中展示 AI 对话界面。可被嵌入到任何 Web 应用中。

2.6 mom — Slack Bot

Slack 机器人，将 Slack 消息委托给 pi coding agent 处理。说明 pi 的核心可以被不同前端（TUI、Web、Slack）包裹。

2.7 pods — vLLM 部署

CLI 工具，用于在云端管理 vLLM 部署。属于周边功能。

第三部分：核心实现流程深入剖析

3.1 用户输入到 LLM 响应的完整流程

用户输入 "Fix the login bug"

|

▼

AgentSession.prompt(text, options)

- |— 1. Extension 命令检查 (/开头? Extension 已注册该命令?)
- |— 2. Input 事件 (extension 可拦截/转换)
- |— 3. Skill 展开 (/skill:name → 读取 SKILL.md 注入)
- |— 4. Prompt Template 展开 (/template → 替换变量)
- |— 5. Streaming 检查 (如果 agent 在运行, 排入 steer/followUp 队列)
- |— 6. Flush 待发 bash 消息
- |— 7. Model + API Key 验证
- |— 8. 构建 messages 数组 (用户消息 + 下一轮待发自定义消息)
- |— 9. before_agent_start 扩展事件 (可注入消息 + 修改系统提示)
- |— 10. agent.prompt(messages)

|

▼

Agent._runLoop(messages)

- |— 创建 AgentLoopConfig (含 model, convertToLlm, transformContext, ...)
- |— agentLoop(messages, context, config, signal, streamFn)

|

▼

runLoop() 外循环 [处理 followUp]

└─ 内循环 [处理工具调用 + steering]

- |— streamAssistantResponse()
 - | |— transformContext() - 上下文转换
 - | |— convertToLlm() - AgentMessage[] → Message[]
 - | |— 构建 LLM Context { systemPrompt, messages, tools }
 - | |— getApiKey() - 解析 API Key (支持过期令牌动态刷新)
 - | |— streamFunction(model, llmContext, options)
 - | |— provider.streamSimple() - 调实际 API
- |— 检查 toolCalls
 - | |— executeToolCalls()
 - | | |— 对每个 toolCall:
 - | | |— tool_execution_start 事件
 - | | |— validateToolArguments(tool, toolCall)
 - | | |— tool.execute(id, args, signal, onUpdate)
 - | | |— tool_execution_end 事件


```

    |
    |   └─ 检查 getSteeringMessages() – 有则中断
    |   └─ 返回 { toolResults, steeringMessages }
    |
    └─ turn_end 事件
|
▼
AgentSession._handleAgentEvent()
├─ 更新 steering/followUp 队列
├─ 转发到 Extension Runner (26种事件)
├─ 发射到外部监听器
├─ 持久化到 SessionManager (JSONL)
├─ 检查可重试错误 → 自动重试 (指数退避)
└─ 检查上下文压缩 → 自动压缩 (阈值/溢出)

```

3.2 Extension 加载流程

```

AgentSession._buildRuntime()
├─ 创建内置工具 (read, bash, edit, write, ...)
├─ ResourceLoader.getExtensions()
|   └─ Loader 发现 + import() Extension 模块
|       └─ 调用 ExtensionFactory(pi: ExtensionAPI)
|           └─ Extension 通过 pi.on(), pi.registerTool(), etc 注册
├─ 创建 ExtensionRunner (管理所有加载的 Extension)
├─ bindCore() (注入 actions + context actions)
├─ 注册 Extension 工具到工具注册表
├─ wrapToolsWithExtensions() (让 Extension 可拦截所有工具调用)
└─ 构建系统提示 (含工具描述)

```

3.3 Session 树结构 (JSONL)

pi 的会话存储使用 JSONL 格式的**树结构**，每条记录有 id 和 parentId：

```

Entry(id=1, parentId=null, type="message", role="user")
├─ Entry(id=2, parentId=1, type="message", role="assistant")
|   └─ Entry(id=3, parentId=2, type="message", role="user") ← 分支 A
|       └─ ...
└─ Entry(id=4, parentId=2, type="message", role="user") ← 分支 B (fork from id=
    └─ ...

```

这实现了不创建新文件的原地分支—— /tree 命令在单文件中导航整个对话历史树， /fork 创建新文件。

第四部分：与 oh-my-opencode / OpenCode 详细对比

4.1 架构定位对比

维度	pi-mono	oh-my-opencode	OpenCode
本质	独立完整产品	OpenCode 插件	独立 CLI 编程助手
包结构	7 包 monorepo	单包 + 子包	单体应用
代码量	~30k LOC (估)	~143k LOC	~50k LOC (Go)
语言	TypeScript (npm)	TypeScript (Bun)	Go
LLM API	自研 pi-ai	使用 OpenCode 内置	使用 AI SDK
Agent 运行时	自研 pi-agent-core	使用 OpenCode 内置	使用 AI SDK
TUI	自研 pi-tui	使用 OpenCode 内置	使用 Bubble Tea
扩展性	Extension System	Plugin + Hook 系统	Plugin 接口

关键差异：pi 拥有整个技术栈，oh-my-opencode 寄生在 OpenCode 上。

4.2 Agent 系统对比

维度	pi-mono	oh-my-opencode
Agent 数量	1 个 (核心 Agent)	11 个专用 Agent
Agent 粒度	通用，通过 Extension 特化	每个 Agent 专精一个领域
子代理	不内置 (Extension 实现)	内置 Prometheus→Atlas 编排
Agent 模型选择	用户手动选择/循环	三级 fallback + Category 模型映射
Plan/Build	不内置	内置 Prometheus(Plan) → Atlas(Build)
多模态	支持 (图片输入)	Multimodal-Looker Agent

pi 的理念：一个通用 Agent + Extension 特化。oh-my-opencode 的理念：专用 Agent 矩阵。

oh-my-opencode 的 11 个 Agent：

- Sisyphus（通用）、Hephaestus（代码工匠）、Oracle（代码审查）
- Librarian（文档）、Explore（代码探索）、Atlas（精确构建）
- Prometheus（规划）、Metis（智慧）、Momus（评审）
- Multimodal-Looker（多模态）、Sisyphus-Junior（轻量）

对比分析：

- pi 的单 Agent 方式更**简单直接**——用户和模型一对一对话
- oh-my-opencode 的多 Agent 方式更**有针对性**——不同任务自动选择最合适的 Agent+模型组合
- 但 pi 通过 Extension 可以实现类似的多 Agent 编排（有人已用 Extension 做了 plan mode）

4.3 工具系统对比

维度	pi-mono	oh-my-opencode	OpenCode
内置工具数	7（默认启用 4）	26	~10
工具类型	文件操作 + bash	文件 + 代理 + 技能 + 后台 + tmux + ...	文件 + bash + 搜索
工具可定制	Extension registerTool()	ToolRegistry + 配置	有限
工具拦截	tool_call / tool_result 事件	tool.execute.before / after hooks	无
Schema	TypeBox	Zod v4	Zod

pi 的工具数量极少但每个都经过精心打磨（如 edit-diff.ts 的模糊匹配）。oh-my-opencode 工具丰富但更复杂：

oh-my-opencode 的 26 个工具包括：

- delegate-task（子代理委派）、start-work（Plan/Build 入口）
- skill 相关（skill-executor、skill-loader）
- 后台代理（background-agent 相关）
- tmux 工具
- 标准文件操作（read、write、edit、search、grep、ls、glob）

4.4 Hook / Extension 系统对比

维度	pi Extension	oh-my-opencode Hook	OpenCode Plugin
事件类型	26 种	46 种	8 种 handler
可注册	工具/命令/快捷键/Flag/Provider/消息渲染器	工具/Hook	工具/Hook
UI 控制	完全（替换编辑器、自定义组件、overlay）	无（纯后端）	无
运行位置	进程内	同进程	同进程
语言	TypeScript	TypeScript	TypeScript
分发	pi package（npm/git）	内置	内置
模型注册	registerProvider()（含 OAuth）	通过 config handler	无

pi 的 Extension 系统明显更强大——它可以修改 UI 的每一个部分、注册新的 LLM 提供商、注册自定义命令和键盘快捷键。oh-my-opencode 的 Hook 系统更专注于**数据流拦截**（消息转换、工具拦截、规则注入）。

4.5 用户界面对比

维度	pi-mono	oh-my-opencode / OpenCode
TUI 框架	自研 pi-tui	Ink（React for CLI）→ 后迁移到自建
渲染	差异渲染 + CSI 2026	全量渲染
组件	15+ 内置组件	功能组件
图片	内联支持（Kitty/iTerm2）	无
主题	可自定义主题（热重载）	有限定制
Editor	自研多行编辑器 + Tab 补全	基础编辑器
Session 浏览	/tree 树导航 + /fork 分支	会话列表
快捷键	完全可配置 + Extension 可注册	固定快捷键

维度	pi-mono	oh-my-opencode / OpenCode
Web UI	有（web-ui 包）	无
RPC 模式	有（进程间通信）	无

4.6 Session / 上下文管理对比

维度	pi-mono	oh-my-opencode	OpenCode
存储格式	JSONL 树结构	由 OpenCode 管理	SQLite
分支	原地分支（/tree 导航）	无（线性）	无
压缩	自动/手动，Extension 可替换	使用 OpenCode 内置	简单截断
导出	HTML 导出 / Gist 分享	无	无
恢复	-c 继续 / -r 浏览 / --session	自动恢复	自动恢复
label/书签	支持	无	无

pi 的 Session 树是一个杀手级特性——你可以在任意历史节点创建分支，所有分支共存于单一文件中。

4.7 提供商 / 模型支持对比

维度	pi-mono	oh-my-opencode	OpenCode
内置提供商	17+	通过 OpenCode 配置	多个
OAuth 登录	/login（多个提供商）	有限	无
自定义提供商	models.json + Extension registerProvider()	配置文件	配置文件
模型数据库	自动生成（models.generated.ts）	OpenCode 管理 + 配置覆盖	配置管理
费用追踪	内置（精确到 cache_read/write）	由 OpenCode 提供	无
WebSocket 传输	支持（Codex）	不支持	不支持

4.8 设计哲学对比

原则	pi-mono	oh-my-opencode / OpenCode
核心复杂度	最小化核心，Extension 扩展	丰富内置功能
子代理	反对内置	大量使用（11 Agent）
MCP	反对（"build CLI tools with READMEs"）	三层 MCP 系统
Plan/Build	反对内置	内置 Prometheus→Atlas
权限控制	反对弹窗（"run in a container"）	无内置，依赖 OpenCode
工具数量	极少（4 默认）	极多（26 工具）
自研 vs 复用	全栈自研	寄生在 OpenCode 上

4.9 优劣势分析

pi-mono 优势

- 1. **技术栈独立**：不依赖任何外部 Agent 框架，从 LLM API 到 TUI 全部自研，升级/修复不受上游限制
- 2. **Extension 系统极其强大**：可以修改 UI 的每一个方面，注册提供商、工具、命令、快捷键——真正的"平台"
- 3. **Session 树**：原地分支 + 树导航是独有的杀手级特性
- 4. **极简内核**：Agent 循环只有 5 个文件，理解和贡献门槛极低
- 5. **多模式运行**：交互 / 打印 / JSON / RPC / SDK——一套核心多种前端
- 6. **费用追踪**：精确的 token 用量和费用计算
- 7. **Steering + FollowUp 消息队列**：用户可以在 Agent 工作时排队发送消息
- 8. **社区生态**：17.5k stars, 127 contributors, 160 releases, pi package 生态
- 9. **WebSocket 传输**：支持 Codex 等新一代 API 传输
- 10. **自研 TUI**：差异渲染、内联图片、IME 支持——终端体验极佳

pi-mono 劣势

- 1. **无内置子代理编排**：复杂任务（如"先调研后实现"）需要用户手动或安装第三方 Extension
- 2. **无内置 Plan/Build**：大型重构任务没有结构化的规划→执行流程
- 3. **无 MCP 支持**：不能直接连接 MCP 服务器（需 Extension）
- 4. **学习曲线**：Extension API 虽然强大但有 26 种事件类型需要理解
- 5. **npm 生态**：Node.js 运行时开销，冷启动较慢

6. 工具精简可能过度：默认只有 4 个工具，某些场景（如 grep 大仓库）需要手动启用

oh-my-opencode 优势

- 1. 丰富的内置功能：11 Agent + 46 Hook + 26 工具——开箱即用
- 2. 多 Agent 编排：Prometheus→Atlas Plan/Build 模式是独特的差异化
- 3. 三层 MCP：内置 + Claude Code + Skill 嵌入——MCP 支持最完善
- 4. Category 模型映射：自动为不同任务类型选择最合适的模型，无需用户干预
- 5. 配置层级：项目级 → 用户级 → 默认值，支持 JSONC 注释
- 6. Bun 运行时：极快的启动和构建速度
- 7. Skill 系统：内置 Skill 加载器 + MCP 管理器

oh-my-opencode 劣势

- 1. 依赖 OpenCode：插件架构意味着受宿主限制（只有 8 个 plugin handler 可用）
- 2. 无自主 UI 控制：不能修改终端 UI，完全依赖 OpenCode 的渲染
- 3. 复杂度高：143k LOC / 1208 文件，贡献门槛高
- 4. 无 Session 树：线性会话，无分支导航
- 5. 无 SDK / RPC 模式：不能作为库嵌入其他应用
- 6. 无独立提供商注册：依赖 OpenCode 的提供商配置

4.10 使用场景推荐

场景	推荐方案	理由
日常编码助手	pi-mono	轻量、快速、Extension 生态丰富
大型项目重构	oh-my-opencode	Plan/Build 模式 + 多 Agent 编排
自定义工作流	pi-mono	Extension 可修改一切
团队标准化	均可	pi 通过 pi package 分发；oh-my-opencode 通过 JSONC 配置
嵌入其他应用	pi-mono	有 SDK + RPC 模式
MCP 生态整合	oh-my-opencode	三层 MCP 支持
最大化定制 UI	pi-mono	Extension 可替换编辑器/页脚/页头/overlay

场景	推荐方案	理由
历史分支管理	pi-mono	Session 树 + /tree + /fork
智能模型选择	oh-my-opencode	Category→Model 自动映射
纯后端/CI 使用	pi-mono	pi -p 打印模式 + JSON 模式
协作开发	pi-mono	Gist 分享 + HTML 导出

4.11 总结

pi-mono 和 **oh-my-opencode** 代表了 AI 编码工具的两种截然不同的设计哲学：

pi-mono = 极简内核 + 无限扩展

- 核心只做一件事做好：LLM 对话 + 工具调用
- 一切高级功能通过 Extension 实现
- 控制整个技术栈（LLM API → Agent → TUI）
- Session 树是独特创新

oh-my-opencode = 丰富内置 + 智能编排

- 开箱即用的 11 Agent + Plan/Build + MCP
- 自动化程度高（模型选择、任务分配）
- 但受限于 OpenCode 的插件边界
- 复杂度换来了开箱即用的强大功能

如果用一句话总结：**pi-mono 像 Vim（极简核心 + 无限插件），oh-my-opencode 像 VS Code（丰富内置 + 可配置）**。选择取决于你是想要"一套工具适配所有工作流"（pi）还是"开箱即用的智能编码代理"（oh-my-opencode）。